APPENDIX A
PROOF

We first describe the uncompressed network model in § A-A. Next, in § A-B, we introduce the network compression model and prove its equivalence to the original network, meaning that for every uncompressed route, there exists a corresponding compressed route that can be mapped back to it, and vice versa.

A. Uncompressed Network Model

Stable Routing Problem (SRP) [1]. An SRP formally captures the set of all routing behaviors a network can exhibit. As illustrated in Figure 1, an SRP instance is defined as a tuple $\langle G, A, a_d, \prec, trans\rangle$. Here, $G = (V, E, d)$ is a directed graph, where V is the set of nodes (routers), $E \subseteq V \times V$ is the set of directed edges (links), and $d \in V$ is the designated destination node. Note that an edge $e = (u, v)$ indicates that node v can receive routing information from node u, while data packets would flow from v to u. A is the set of routing attributes, which describe the content of routing messages(*e.g.*, in BGP: local preference, community tags, and AS path). We define $A_\perp = A \cup \{\perp\}$, where $\perp$ denotes the absence of a valid route (*i.e.*, no routing information). The special attribute a_d represents the original route originated by the destination node d. The partial order $\prec$ over A defines how routers compare route attributes to select the best route(e.g., BGP compares local preference, then AS path length, and so on). For instance, if $a_1 \prec a_2$, then a_1 is considered more preferred than a_2. Finally, $trans$ is a transfer function that models how attributes are transformed as they traverse edges. In BGP, this captures the application of export policies and import policies based on local configuration. Given an edge and an incoming attribute, the transfer function determines the resulting attribute yielded at the receiving node.

SRP Solutions. Given an SRP instance, a solution represents one possible stable routing outcome in the network. Intuitively, a solution ensures that each node is locally stable, that is, it has no reason to switch to a different neighbor's route because the current one is already among the best available. In BGP, a solution for each router represents its current best route. An SRP may have multiple stable solutions depending on topology, attributes, and the comparison relation.

Formally, a solution is a labeling function $L : V \to A$ that assigns each node a final routing attribute, indicating the route it will use to reach the destination. This labeling must meet the following constraints:

- The destination node d must be labeled with a_d, which represents the original route it advertises;
- If a node u receives no valid attributes from its neighbors ($attrs_{L(u)} = \emptyset$), then it has no route ($\perp$) to the destination;
- Otherwise, the node chooses its route $L(u)$ as one of the most preferred attributes in $attrs_L(u)$, based on the partial order $\prec$. If there are multiple equally preferred options (denoted $\approx$), any of them can be selected. (Figure 1, SRP solution)

B. Compressed Network Model

Then we introduce a network compression model—an abstracted SRP, similar to Bonsai [1]—which we denote as $\widehat{SRP}$ (Figure 1, Compression Model). Our goal is to establish routes- equivalence, showing that every route in the original SRP has an equivalent abstraction in $\widehat{SRP}$ can be decompressed to the original one, provided certain abstract conditions are met.

1) Network Compression Definition.

We define a mapping f from physical devices to logical nodes, written as $u \mapsto \widehat{u}$, where each device u maps to a compressed node $\widehat{u}$. Similarly, we define a mapping h from physical routes to logical routes with the addition of two new attributes in $\widehat{A}$. Since in BGP, a router either originates a route itself or learns a route from a neighbor via a link, we associate each route a with an edge $e_a = (u, v)$, where a is sent by u and received by v (or $e_a = (\varnothing, d)$ for an originated route). Based on this, we define the mapping function as $h(e_a, a) = \widehat{a}$. However, this definition makes it difficult to fully express cases such as

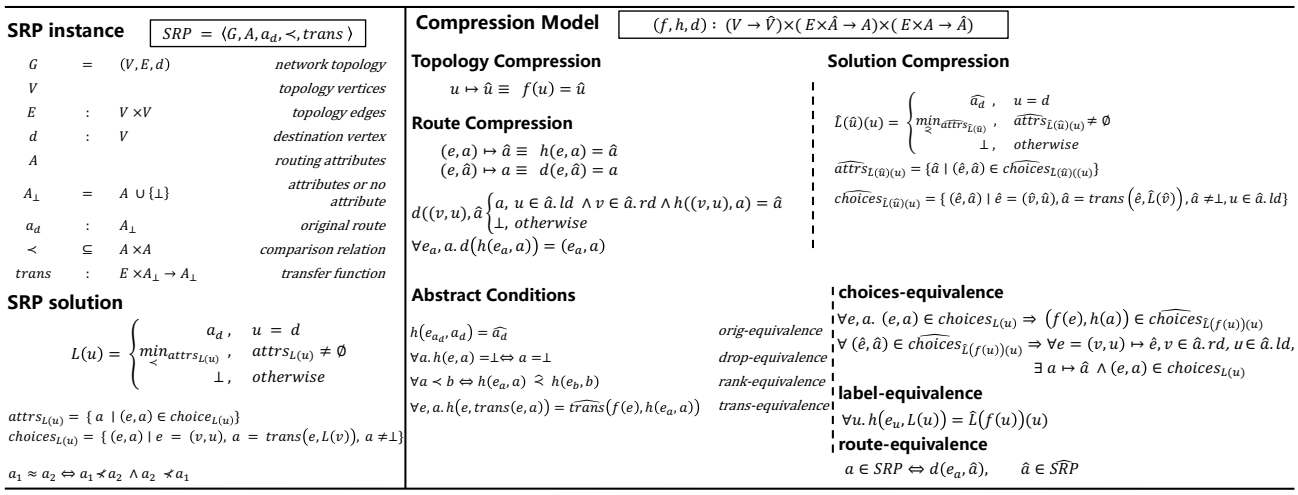

Fig. 1: Definitions of SRPs, solutions, abstract conditions, and equivalent properties.

the example $\widehat{a}_d = (100, \Phi, \langle\rangle, [D_1], [\,])$. For device D_1, it is clear from $\widehat{a}_d$ that the original route is $a = (100, \Phi, \langle\rangle)$, but for D_2, we also want to represent that the original route is $\perp$. This is not easily expressed using only h. To better capture this mapping, we introduce a decompression function d (Figure 1, Route Compression). This function satisfies the relationship $d(h(e_a, a)) = (e_a, a)$, enabling us to accurately recover each original route from the compressed form.

2) Solution Compression

When we compress the destination device together with other devices, it may lead to a situation where the destination device selects its original route as the best route, while other devices within the same logical node may choose different routes received from their neighbors (this issue can also occur when devices have different AS numbers). To address this, we define $\widehat{L}(\widehat{u})(u)$ (Figure 1, Solution Compression), such that each physical device u in the logical node $\widehat{u}$ independently selects its own best route. The result is still represented in a logical form, since some devices may still choose the same logical route as their optimal choice.

3) Abstract Conditions.

The abstract conditions require that the mapping f and h satisfy the following four properties:

1. Orig-equivalence: Each original route generated by a device must map to a corresponding logical original route;
2. Drop-equivalence: A route dropped in the original network is also dropped in the compressed one, and vice versa;
3. Rank-equivalence: The comparison relation ($\prec$) used to select the best route must be consistent between physical and logical routes;
4. Trans-equivalence: The transfer behavior of a logical route across a logical edge must accurately represent the behavior of all corresponding physical routes across their respective physical edges. (Figure 1, Abstract Conditions)

4) Routes Equivalence

Now we prove that when the abstract conditions are satisfied, routes-equivalence holds by leveraging choices-equivalence and label-equivalence.

Theorem 1. *If $\widehat{SRP}$ satisfies abstract conditions, then it also satisfies choices-equivalence.*

Proof. Suppose there exists an attribute $a = trans(e, L(v)) \neq \perp, e = (v, u)$ in the original network. Then, by the definition of choices, we have $(e, a) \in choices_{L(u)}$. By the drop-equivalence, $h(a) \neq \perp$. By the trans-equivalence, we know: $h(e, trans(e, L(v))) = \widehat{trans}(f(e), \widehat{L}(f(v)(v))$. Hence, there exists $\widehat{a} = \widehat{trans}(f(e), \widehat{L}(f(v)(v)) \neq \perp$, which implies that $(f(e), h(e, a)) \in \widehat{choices}_{\widehat{L}(f(u))(u)}$. Conversely, suppose $\widehat{a} = \widehat{trans}(\widehat{e}, \widehat{L}(f(v)(v)) \neq \perp$ is an attribute in the compressed network. By trans-equivalence, we know: $\widehat{trans}(\widehat{e}, \widehat{L}(f(v)(v)) = h(e, trans(e, L(v)))$. Let $a = trans(e, L(v)) \neq \perp$, then we have $h(e, a) = \widehat{a}$ and $v \in \widehat{a}.rd, u \in \widehat{a}.ld$. Thus, $(e, a) \in choices_{L(u)}$. Therefore, the set of choices is preserved under the mapping, completing the proof of choices-equivalence. □

We now prove that for every node u, the selected route in the original network $L(u)$ and its mapped logical route $\widehat{L}(f(u)(u))$ in the compressed network are equivalent under mapping h; that is, $h(e_u, L(u)) = \widehat{L}(f(u)(u))$.

Theorem 2. *If $\widehat{SRP}$ is choices-equivalent, then it is also label-equivalent.*

Proof. For any node u, $L(u)$ has three possible cases. If $u = d$ is the destination node, then $L(u) = a_d$. By the definition, $h((\phi, d), a_d) = \widehat{a}_d$, which satisfies $\widehat{L}(f(d)) = \widehat{a}_d = h((\phi, d), a_d)$. If u has no valid routes, meaning $attrs_{L(u)} = \emptyset$, then $L(u) = \perp$. By the drop-equivalence property, we have $h(e_u, L(u)) = \perp$, and from choices-equivalence, $\widehat{attrs}\widehat{L}(f(u)(u)) = \emptyset$, thus $\widehat{L}(f(u)(u)) = \perp = h(e_u, L(u))$.

Now we prove the third case: when $L(u) = a \neq \perp$, and a is selected as the most preferred attribute at u under $\prec$.

($\Rightarrow$) Direction: If $L(u) = a$, then $a \in attrs_{L(u)}$ is the $\prec$-minimal element, and thus $(e, a) \in choices_{L(u)}$ for some edge e. By the choices-equivalence property, we have $(f(e), h(e, a)) \in \widehat{choices}\widehat{L}(f(u)(u)$. Further, by rank-equivalence, $h(e, a)$ is the $\widehat{\prec}$-minimal element in $\widehat{attrs}_{\widehat{L}(f(u)(u))}$, so, $\widehat{L}(f(u)(u)) = h(e, a) = h(e, L(u))$.

($\Leftarrow$) Direction: If $\widehat{L}(f(u)(u)) = \widehat{a}$, then $\widehat{a} \in \widehat{attrs}_{\widehat{L}(\widehat{u})(u)}$ and is $\widehat{\prec}$-minimal. By choices-equivalence, for any $\widehat{e}$, and any $e = (v, u) \mapsto \widehat{e}, v \in \widehat{a}.rd, u \in \widehat{a}.ld$, there exists $(e, a) \in choices_{L(u)}$ such that $h(e, a) = \widehat{a}$. Among all such a values, let a be the one that is minimal under $\prec$, then $L(u) = a$, and thus, $h(e, L(u)) = h(e, a) = \widehat{a} = \widehat{L}(f(u)(u))$.

Therefore, in all cases, the solution labeling satisfies $\widehat{L}(f(u)(u)) = h(L(u))$, establishing label equivalence. □

Now, we prove routes-equivalence based on choices-equivalence and label-equivalence. Specifically, for every physical route a in the original network, there exists a corresponding compressed route $\widehat{a}$ in $\widehat{SRP}$ that can be faithfully mapped back to a, and vice versa.

Theorem 3. *If $\widehat{SRP}$ is label-equivalent, then it is also routes-equivalent.*

Proof. We prove by induction on the process of routing propagation. Initially, when a destination node generates its own route, orig-equivalence ensures that each physical route a_d directly maps to a corresponding logical route $\widehat{a}_d$ under mapping h. As routes propagate through the network, devices select their best routes from the candidate set. Because label-equivalence guarantees that the comparison relation $\prec$ is preserved, each device and its corresponding logical node select the same best route. When a selected route is propagated along a physical edge (u, v), trans-equivalence ensures that the resulting propagated route is mapped consistently to its logical counterpart across $(\widehat{u}, \widehat{v})$. Consequently, when a neighbor receives candidate routes, choices-equivalence guarantees that its candidate set exactly matches that of its logical node under the mapping h. By induction, since route generation, selection, propagation, and reception are all equivalent at every step, for every physical route a constructed in SRP, there exists a compressed route $\widehat{a}$ in $\widehat{SRP}$ with $h(a) = \widehat{a}$. Because of $d(h(e_a, a)) = (e_a, a)$, any compressed route $\widehat{a}$ can also be mapped back to its original route a, establishing that SRP and $\widehat{SRP}$ are routes-equivalent. □

From Theorems 1, 2, and 3, we conclude that routes-equivalence holds when the abstract conditions are satisfied.

APPENDIX B
ALGORITHMS FOR RIP AND OSPF

We first present the compression algorithms in § B-A, then present the simulation algorithm in § B-B for other protocols (RIP and OSPF).

A. Compression Algorithms

RIP (distance vector). RIP is a distance-vector protocol that selects shortest paths based on hop count. Accordingly, each device uses the same transfer function: it increments the hop count when advertising a route and drops the route if the hop-count limit is exceeded. Therefore, for RIP, our compression only needs to satisfy the Edge-constraint. Since RIP has no export or import policies, devices within the same logical node only need to share the same neighbors. Under these conditions, all devices in a logical node share the same solution behavior (selecting routes by hop count) and the same transfer behavior (incrementing hop count and enforcing the hop-count limit).

OSPF (link state). Open Shortest Path First (OSPF) is a link-state protocol in which routers exchange link-cost information and compute least-cost paths to the destination. The transfer function adds the configured link cost. Thus, for OSPF, our compression again only needs to satisfy the Edge-constraint. Since OSPF has no export or import policies, devices within the same logical node must have the same neighbors and the same link costs. Under these conditions, all devices in a logical node share the same solution behavior (selecting least-cost paths) and the same transfer behavior (adding the same link cost).

B. Simulation Algorithm

RIP (distance vector). Because RIP uses hop count as a nonnegative metric, the simulation yields the same result with either Dijkstra's or Bellman–Ford, making our simulation algorithm directly applicable to RIP.

OSPF (link state). Our simulation algorithm is based on Dijkstra's algorithm, and thus naturally applies to link-state protocols such as OSPF.

APPENDIX C
VERIFICATION ALGORITHM

A. Single-Destination Data-Plane Verification

As shown in Algorithm 1, ***sNetVeri*** first encodes packet spaces into BDDs. It collects all route prefixes and builds a prefix-to-BDD cache (lines 8–12). Then, for each node, ***sNetVeri*** groups routes by next-hop neighbor, which corresponds to an output port, and unions the packet spaces associated with each port and destination spaces(lines 15–17 and lines 22). For each target node, ***sNetVeri*** additionally records per-physical-device packet spaces and destination spaces, so that the final reachability result can be mapped back to physical devices (lines 18–25).

After encoding, sNetVeri performs BFS over the compressed topology, starting from the destination node. It maintains a queue of nodes together with their currently reachable packet space predicates, as well as a visited map that records the packet space already explored on each port (lines 28–29). For each popped node n and predicate pred (lines 31), ***sNetVeri*** traverses each neighbor v. It first checks whether port (n, v) has already been visited with a superset of the current predicate, in which case the traversal can be pruned (lines 32–36). Otherwise, ***sNetVeri*** computes the packet space that can be forwarded to v. If v is a target node, ***sNetVeri*** records the reachable packet space for later physical-device-level checks (lines 37 and 40–50). If the remaining packet space is non-empty, ***sNetVeri*** pushes v and its reachable predicate into the queue (line 38), and then updates the visited map for the port (line 39). The algorithm terminates when the queue becomes empty.

B. Multi-Destination Data-Plane Verification

As shown in Algorithm 2, ***sNetVeri*** first encodes the packet spaces for all nodes (line 2). It then constructs a single Net for each destination that covers all source nodes, and uses the destination packet space to prune unreachable packet spaces during graph construction (line 5). Each destination-specific Net is verified independently on the compressed network, enabling parallel execution across destinations (line 6). Finally, to determine reachability for each physical destination–source pair, ***sNetVeri***

Algorithm 1: Single-Destination Data-Plane Verification

```
1  Function ENCODING(G, Nodes, Targets, X):
2      P ← ∅ ;                                          // Set of unique route prefixes appearing in the network
3      C_p ← ∅ ;                                                  // Cache from each prefix to its BDD encoding
       // Packet-space maps for physical-device and node-level forwarding
4      M_ps^phy ← ∅;
5      M_ps^n ← ∅;
       // Destination-space maps for node-level and physical-device reachability
6      M_ds^n ← ∅;
7      M_ds^phy ← ∅;
8      foreach n ∈ N do
9          foreach route ∈ n do
10             P ← P ∪ {(route.prefixIp, route.prefixLen)};
11     foreach (ip, len) ∈ P do
12         C_p[len][ip] ← ENCODEPREFIXBDD(ip, len, X);
13     foreach n ∈ Nodes do
14         foreach route ∈ n do
15             nhn ← route.nextHopNode;
16             b ← C_p[r.len][r.ip];
17             M_ps^n[n][nhn] ← M_ps^n[n][nhn] ∨ b ;
18             if n ∈ Targets then
19                 foreach ld ∈ n.localDevices do
20                     M_ps^phy[n][ld][nhn] ← M_ps^phy[n][ld][nhn] ∨ b;
21             if ISORIGINALROUTE(route) then
22                 M_ds^n[n] ← M_ds^n[n] ∨ b;
23                 if n ∈ Targets then
24                     foreach ld ∈ n.localDevices do
25                         M_ds^phy[n][ld] ← M_ds^phy[n][ld] ∨ b;
26     return M_ps^phy, M_ps^n, M_ds^n, M_ds^phy ;
27 Function VERIFICATION(Targets, Net, M_ds^n[dst]):
28     queue ← {(Net.dst, M_ds^n[Net.dst])};
29     visited ← ∅ ;
30     while queue ≠ ∅ do
31         (n, pred) ← pop(queue);
32         foreach v ∈ Net.E(n) do
33             key ← (n, v);
34             if key ∈ visited then
35                 if pred ∧ ¬visited[key] = ∅ then
36                     continue
37             if CHECK(n, v, pred, Targets) then
38                 push((v, pred ∧ ¬visited[key]), queue))
39             visited[key] ← visited[key] ∨ v.packetSpace[n]
40 Function CHECK(n,v, pred_current,Targets):
41     reachable ← false;
42     pred ← v.packetSpace[n] ∧ pred_current;
43     v.packetSpace[n] = ¬pred_current ∧ v.packetSpace[n];
44     if pred = ∅ then
45         reachable ← false;
46     if v ∈ Targets then
47         v.reachPreds[n] ← v.reachPreds[n] ∨ pred;
48         reachable ← false;
49     reachable ← true;
50     return reachable;
```

intersects the reachability records of each target node with the destination space of the physical destination device and the packet space of the physical source device (lines 7–13).

Algorithm 2: Multi-Destination Data-Plane Verification

Function PARALLEL VERIFICATION(G,$Nodes$,$Targets$,X):
    $M_{\text{ps}}^{phy}, M_{\text{ps}}^{n}, M_{\text{ds}}^{n}, M_{\text{ds}}^{phy} \leftarrow$ ENCODING$(G, Nodes, Targets, \mathcal{X})$;
    $reachable \leftarrow \emptyset$;
    foreach $dst \in Target$ ***in parallel*** **do**
        $Net \leftarrow$INITTOPOLOGY$(M_{\text{ps}}^{n}, M_{\text{ds}}^{n}[dst])$;
        VERIFICATION$(Targets, Net, M_{\text{ds}}^{n}[dst])$;
        foreach $phy_{dst} \in dst$ **do**
            foreach $src \in Targets$ **do**
                foreach $phy_{src} \in src$ **do**
                    foreach $key_{port} \in src.reachPred$ **do**
                        if $src.reachPred[key_{port}] \wedge M_{ds}^{phy}[phy_{dst}] \wedge M_{ps}^{phy}[src][phy_{src}][key_{port}] \neq \emptyset$ **then**
                            $reachable \leftarrow \{(phy_{dst}, phy_{src})\}$;
                            break;
    return $reachable$;

REFERENCES

[1] R. Beckett, A. Gupta, R. Mahajan, and D. Walker, "Control plane compression," in *SIGCOMM '18*. ACM, 2018, pp. 476–489.